LAPORAN PRAKTIKUM

Struktur Data

MODUL 4

Singly Linked List



Disusun oleh:

Dharma Chandra Viriya

109082500052

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

Link Repo Github Modul 4 : https://github.com/iamCelott/praktikum-struktur-data-smt3-modul-4

LATIHAN GUIDED MODUL

1. **Guided 1**

Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
  infotype info;
  address next;
};

struct List {
  address first;
};

void createList(List &L) { L.first = NULL; }

address alokasi(infotype x) {
  address P = new elmlist;
  P->info = x;
  P->next = NULL;
  return P;
}

void dealokasi(address &P) {
  delete P;
  P = NULL;
}

void insertFirst(List &L, address P) {
  P->next = L.first;
  L.first = P;
}

void insertLast(List &L, address P) {
  if (L.first == NULL) {
    insertFirst(L, P);
  } else {
    address Q = L.first;
```

```
      while (Q->next != NULL) {
        Q = Q->next;
      }
      Q->next = P;
    }
  }

  void insertAfter(List &L, address P, address Prec) {
    if (Prec != NULL) {
      P->next = Prec->next;
      Prec->next = P;
    }
  }

  void deleteFirst(List &L, address &P) {
    if (L.first != NULL) {
      P = L.first;
      L.first = P->next;
      P->next = NULL;
    }
  }

  void deleteLast(List &L, address &P) {
    if (L.first != NULL) {
      if (L.first->next == NULL) {
        deleteFirst(L, P);
      } else {
        address Q = L.first;
        while (Q->next->next != NULL) {
          Q = Q->next;
        }
        P = Q->next;
        Q->next = NULL;
      }
    }
  }

  void deleteAfter(List &L, address &P, address Prec) {
    if (Prec != NULL && Prec->next != NULL) {
      P = Prec->next;
      Prec->next = P->next;
      P->next = NULL;
    }
  }

  void printInfo(List L) {
    address P = L.first;
    while (P != NULL) {
```

```
      cout << P->info << ' ';
      P = P->next;
    }
    cout << '\n';
  }

  int main() {
    List L;
    address P1, P2, P3, P4, P5;

    createList(L);

    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

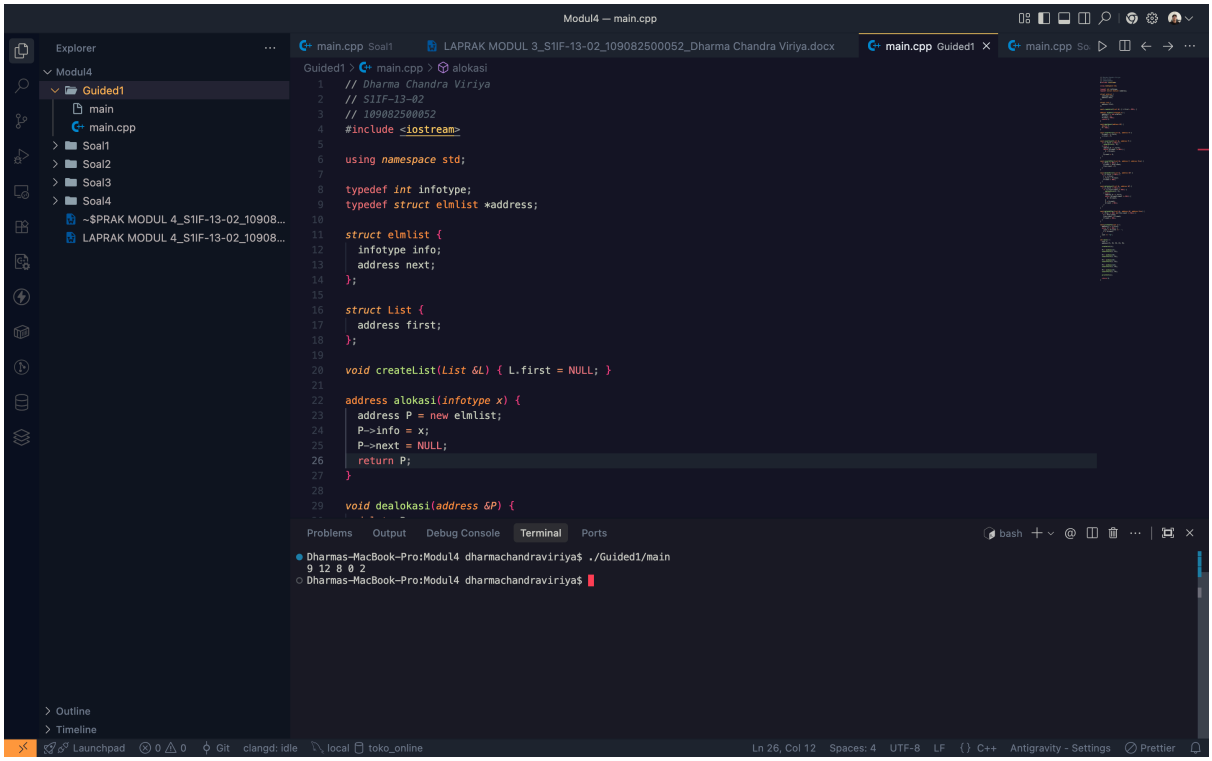
    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    printInfo(L);

    return 0;
  }
```

Screenshoot program



Deskripsi program

Program ini mengimplementasikan Singly Linked List dalam C++ dengan operasi penambahan, penghapusan, dan penampilan data. Struktur elmlist menyimpan nilai integer pada info dan pointer next untuk menghubungkan node, sedangkan List memiliki pointer first sebagai awal list. Fungsi insertFirst() menambahkan node di awal, insertLast() di akhir, insertAfter() setelah node tertentu, sedangkan deleteFirst(), deleteLast(), dan deleteAfter() digunakan untuk menghapus node berdasarkan posisinya.

Pada fungsi main(), lima data, yaitu 2, 0, 8, 12, dan 9, dimasukkan menggunakan insertFirst(). Karena setiap data ditambahkan di awal list, urutannya menjadi 9, 12, 8, 0, 2. Fungsi printInfo() kemudian menelusuri dan menampilkan seluruh data tersebut ke layar.

LATIHAN SOAL MODUL

1. **Soal 1**

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
  string nama;
  Node *next;
};

Node *head = nullptr;
Node *tail = nullptr;

void insertLast(string nama) {
  Node *baru = new Node;
  baru->nama = nama;
  baru->next = nullptr;
  if (head == nullptr) {
    head = baru;
    tail = baru;
  } else {
    tail->next = baru;
    tail = baru;
  }
}

void deleteFirst() {
  if (head == nullptr) {
    cout << "Antrean kosong" << endl;
    return;
  }
  Node *hapus = head;
  cout << "Melayani: " << hapus->nama << endl;
  head = head->next;
  if (head == nullptr) {
    tail = nullptr;
  }
  delete hapus;
}

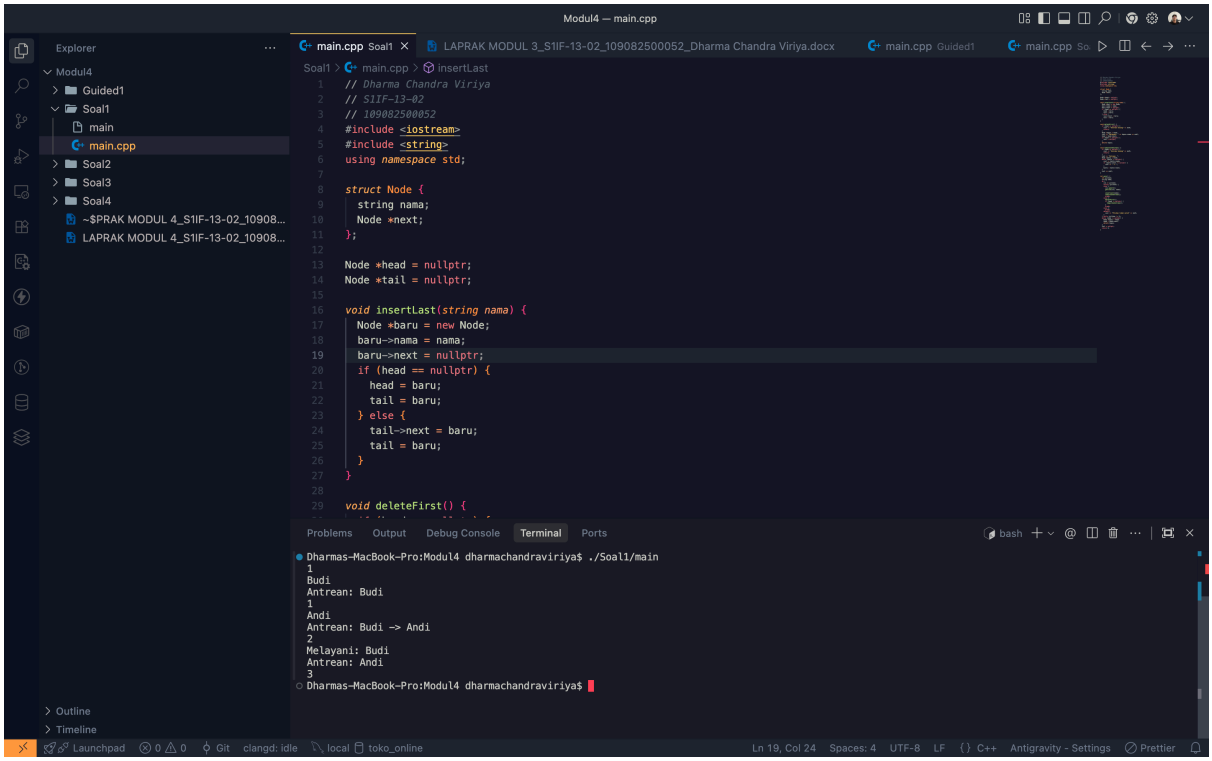
void tampilkanAntrean() {
  if (head == nullptr) {
    cout << "Antrean kosong" << endl;
    return;
  }
```

```
    cout << "Antrean: ";
    Node *bantu = head;
    while (bantu != nullptr) {
      cout << bantu->nama;
      if (bantu->next != nullptr) {
        cout << " -> ";
      }
      bantu = bantu->next;
    }
    cout << endl;
  }

  int main() {
    int pilihan;
    string nama;
    do {
      cin >> pilihan;
      switch (pilihan) {
      case 1:
        cin.ignore();
        getline(cin, nama);

        insertLast(nama);
        tampilkanAntrean();
        break;
      case 2:
        deleteFirst();
        if (head != nullptr) {
          tampilkanAntrean();
        }
        break;
      case 3:
        break;
      default:
        cout << "Pilihan tidak valid" << endl;
      }
    } while (pilihan != 3);
    while (head != nullptr) {
      Node *hapus = head;
      head = head->next;
      delete hapus;
    }
    tail = nullptr;
    return 0;
  }
```

Screenshoot program



Deskripsi program

Program ini menggunakan struktur data Singly Linked List untuk mengelola antrean nasabah bank. Setiap node menyimpan nama nasabah dan pointer next yang menunjuk ke node berikutnya. Pointer head digunakan untuk menunjuk nasabah pertama, sedangkan tail menunjuk nasabah terakhir. Fungsi insertLast() menambahkan nasabah baru di akhir antrean, sementara deleteFirst() melayani nasabah yang berada di urutan pertama dengan menghapus node tersebut dari memori menggunakan delete. Fungsi tampilkanAntrean() digunakan untuk menampilkan seluruh nasabah secara berurutan.

Pada fungsi main(), program menggunakan perulangan do-while dan switch untuk menjalankan menu sampai pengguna memilih angka 3. Setiap kali nasabah ditambahkan atau dilayani, status antrean ditampilkan kembali agar pengguna dapat mengetahui urutan nasabah yang tersisa. Program juga menangani kondisi antrean kosong serta mengatur ulang head dan tail ketika seluruh nasabah sudah dilayani.

2. **Soal 2**

Source Code

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* head = nullptr;

void insertLast(int nilai) {
    Node* baru = new Node;
    baru->data = nilai;
    baru->next = nullptr;
    if (head == nullptr) {
        head = baru;
    } else {
        Node* bantu = head;
        while (bantu->next != nullptr) {
            bantu = bantu->next;
        }
        bantu->next = baru;
    }
}

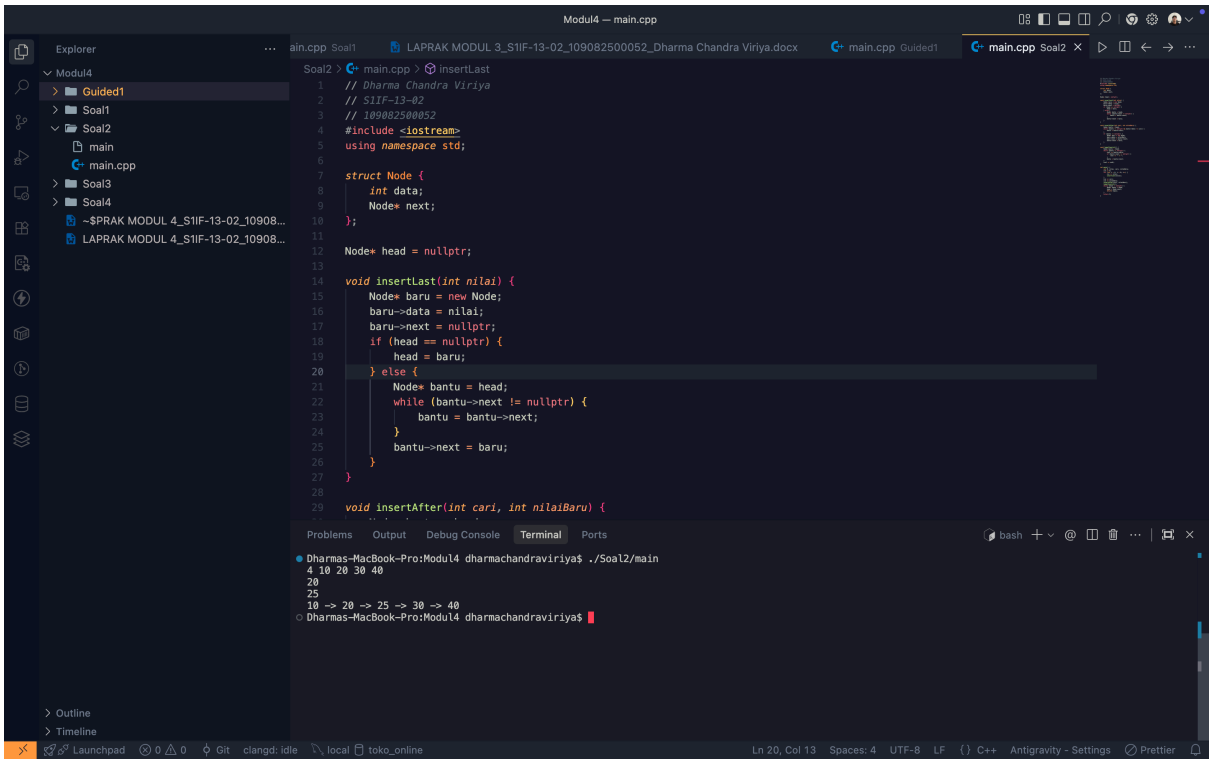
void insertAfter(int cari, int nilaiBaru) {
    Node* bantu = head;
    while (bantu != nullptr && bantu->data != cari) {
        bantu = bantu->next;
    }
    if (bantu != nullptr) {
        Node* baru = new Node;
        baru->data = nilaiBaru;
        baru->next = bantu->next;
        bantu->next = baru;
    }
}

void tampilkanList() {
    Node* bantu = head;
    while (bantu != nullptr) {
        cout << bantu->data;
        if (bantu->next != nullptr) {
            cout << " -> ";
        }
        bantu = bantu->next;
```

```
        }
        cout << endl;
    }

    int main() {
        int N, nilai, cari, nilaiBaru;
        cin >> N;
        for (int i = 0; i < N; i++) {
            cin >> nilai;
            insertLast(nilai);
        }
        cin >> cari;
        cin >> nilaiBaru;
        insertAfter(cari, nilaiBaru);
        tampilkanList();
        while (head != nullptr) {
            Node* hapus = head;
            head = head->next;
            delete hapus;
        }
        return 0;
    }
```

Screenshoot program



Deskripsi program

Program ini menggunakan Singly Linked List untuk menyimpan sekumpulan bilangan bulat. Setiap node memiliki variabel data untuk menyimpan nilai dan pointer next untuk menunjuk node berikutnya. Fungsi insertLast() digunakan untuk memasukkan data awal ke dalam list secara berurutan, sedangkan fungsi insertAfter() bertugas mencari nilai tertentu menggunakan pointer bantu. Pencarian dilakukan dengan menelusuri node satu per satu sampai nilai yang dicari ditemukan atau pointer mencapai nullptr.

Setelah data ditemukan, program membuat node baru dan menyisipkannya tepat setelah node tersebut. Pointer next dari node baru diarahkan ke node berikutnya, kemudian pointer next dari node yang ditemukan diarahkan ke node baru. Fungsi tampilkanList() menampilkan hasil akhir menggunakan tanda panah sebagai pemisah antarelemen. Jika nilai yang dicari tidak ditemukan, list tidak mengalami perubahan.

3. Soal 3

Source Code

```
#include <iostream>
using namespace std;

struct Node {
  int data;
  Node *next;
};

Node *head = nullptr;

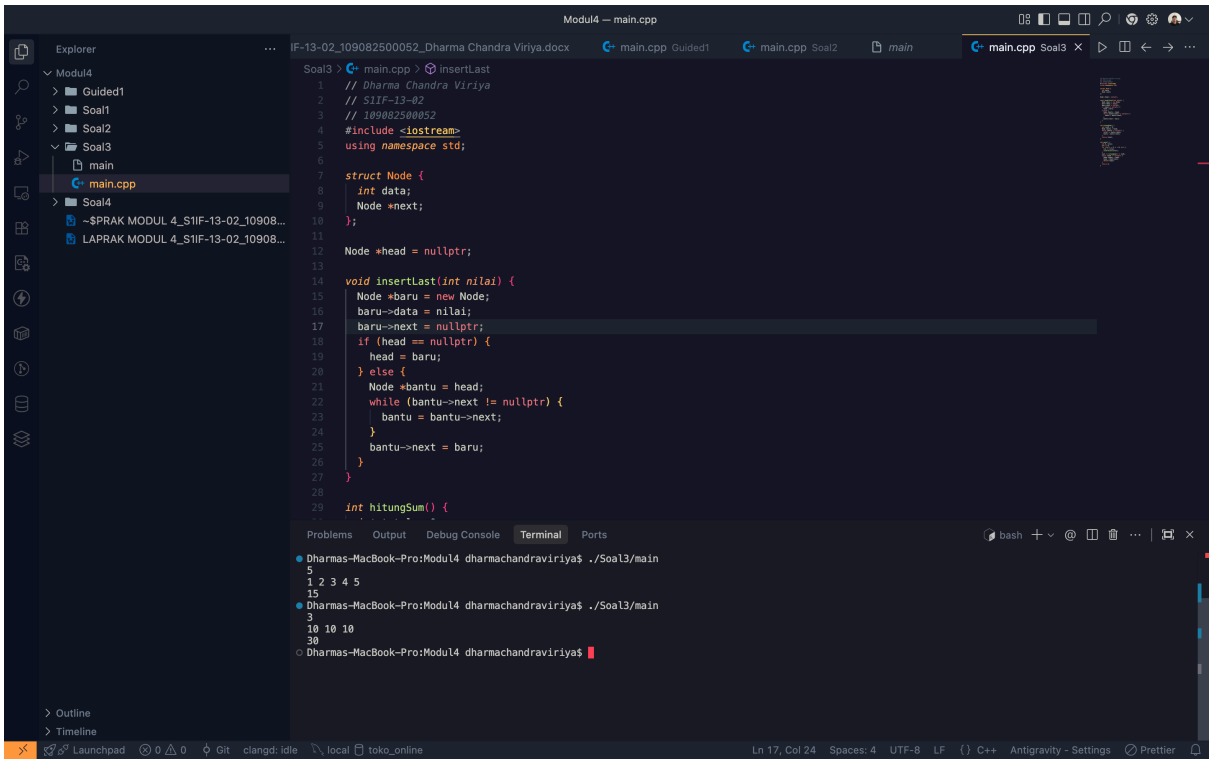
void insertLast(int nilai) {
  Node *baru = new Node;
  baru->data = nilai;
  baru->next = nullptr;
  if (head == nullptr) {
    head = baru;
  } else {
    Node *bantu = head;
    while (bantu->next != nullptr) {
      bantu = bantu->next;
    }
    bantu->next = baru;
  }
}

int hitungSum() {
  int total = 0;
  Node *bantu = head;
  while (bantu != nullptr) {
```

```
        total += bantu->data;
        bantu = bantu->next;
    }
    return total;
}

int main() {
    int N, nilai;
    cin >> N;
    for (int i = 0; i < N; i++) {
        cin >> nilai;
        insertLast(nilai);
    }
    cout << hitungSum() << endl;
    while (head != nullptr) {
        Node *hapus = head;
        head = head->next;
        delete hapus;
    }
    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini menggunakan Singly Linked List untuk menyimpan sejumlah bilangan bulat dan menghitung jumlah seluruh elemennya. Struktur Node terdiri dari variabel data untuk menyimpan bilangan dan pointer next untuk menghubungkan node berikutnya. Fungsi insertLast() memasukkan setiap bilangan ke bagian akhir list, sedangkan fungsi hitungSum() menelusuri seluruh node menggunakan pointer bantu. Setiap nilai yang ditemukan ditambahkan ke variabel total yang awalnya bernilai 0.

Proses penjumlahan terus dilakukan selama pointer bantu tidak bernilai nullptr. Setelah seluruh elemen dikunjungi, fungsi hitungSum() mengembalikan nilai total untuk ditampilkan ke layar. Dengan cara ini, program dapat menghitung jumlah semua elemen tanpa perlu menyimpan data dalam array tambahan. Setelah hasil ditampilkan, seluruh node dihapus dari memori untuk menghindari penggunaan memori yang tidak diperlukan.

4. **Soal 4**

Source Code

```
#include <iostream>
#include <limits>
#include <string>
using namespace std;

struct Node {
  string judul;
  int tahun;
  Node *next;
};

Node *head = nullptr;

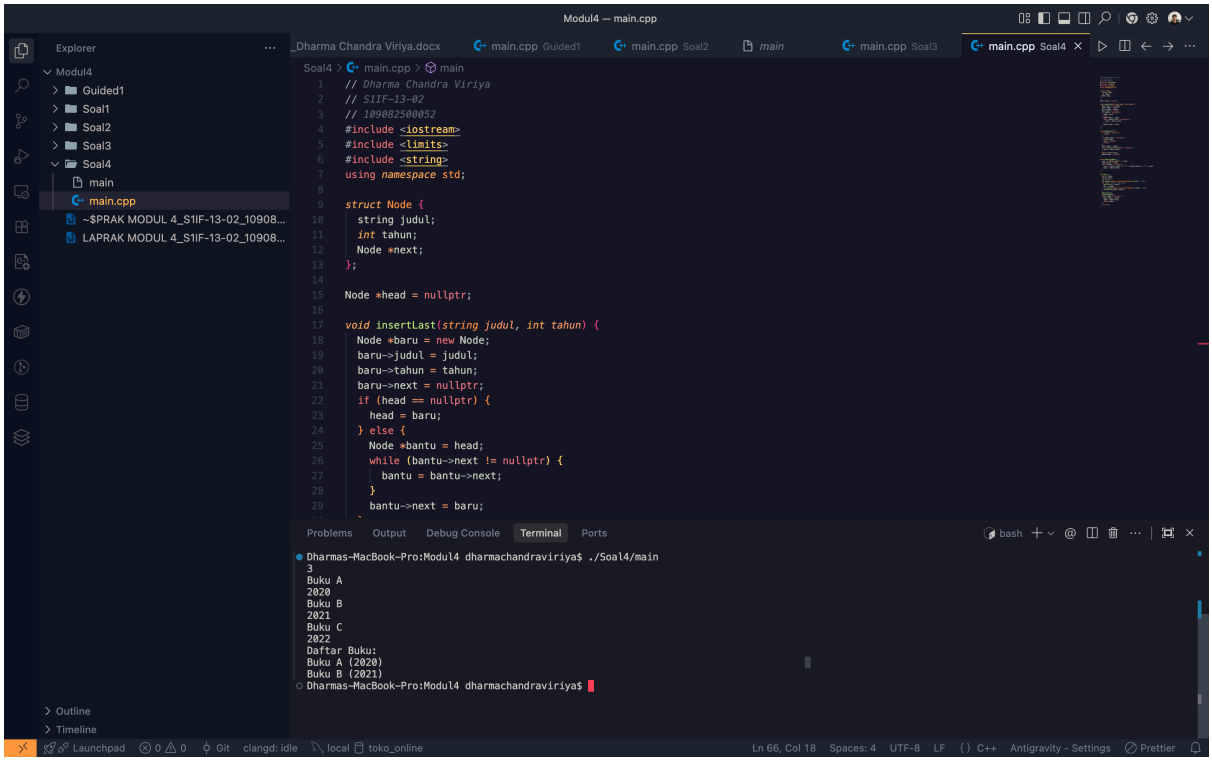
void insertLast(string judul, int tahun) {
  Node *baru = new Node;
  baru->judul = judul;
  baru->tahun = tahun;
  baru->next = nullptr;
  if (head == nullptr) {
    head = baru;
  } else {
    Node *bantu = head;
    while (bantu->next != nullptr) {
      bantu = bantu->next;
    }
    bantu->next = baru;
  }
}
```

```
void deleteLast() {
  if (head == nullptr) {
    return;
  }
  if (head->next == nullptr) {
    delete head;
    head = nullptr;
    return;
  }
  Node *bantu = head;
  while (bantu->next->next != nullptr) {
    bantu = bantu->next;
  }
  delete bantu->next;
  bantu->next = nullptr;
}

void tampilkanBuku() {
  cout << "Daftar Buku:" << endl;
  Node *bantu = head;
  while (bantu != nullptr) {
    cout << bantu->judul << " (" << bantu->tahun << ")" <<
endl;
    bantu = bantu->next;
  }
}

int main() {
  int N, tahun;
  string judul;
  cin >> N;
  cin.ignore(numeric_limits<streamsize>::max(), '\n');
  for (int i = 0; i < N; i++) {
    getline(cin, judul);
    cin >> tahun;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    insertLast(judul, tahun);
  }
  deleteLast();
  tampilkanBuku();
  while (head != nullptr) {
    Node *hapus = head;
    head = head->next;
    delete hapus;
  }
  return 0;
}
```

Screenshoot program



Deskripsi program

Program ini menggunakan Singly Linked List untuk menyimpan data buku perpustakaan yang terdiri dari judul dan tahun terbit. Setiap node memiliki variabel judul bertipe string, tahun bertipe int, serta pointer next yang menghubungkan node dengan buku berikutnya. Fungsi insertLast() digunakan untuk memasukkan data buku ke akhir list. Pada fungsi main(), getline() digunakan untuk membaca judul buku yang dapat mengandung spasi, sedangkan cin digunakan untuk membaca tahun terbitnya.

Fungsi deleteLast() digunakan untuk menghapus buku terakhir dalam list. Program terlebih dahulu memeriksa apakah list kosong atau hanya memiliki satu node. Jika terdapat lebih dari satu node, pointer bantu menelusuri list sampai menemukan node sebelum node terakhir. Node terakhir kemudian dihapus menggunakan delete dan pointer next dari node sebelumnya diatur menjadi nullptr. Setelah proses penghapusan selesai, fungsi tampilkanBuku() menampilkan daftar buku yang masih tersisa.